

TICKETIA

30 Preguntas del Tribunal TFM + Respuestas Técnicas

Preparación completa para la defensa académica

Trabajo Fin de Master — Ingeniería de Sistemas IA | 2026

Nivel	Descripcion	Preguntas
COMPRENSION	Conceptos generales, decisiones de diseno, flujos basicos	1 – 10
TECNICA	Implementacion, patrones, seguridad, calidad del codigo	11 – 20
AVANZADA	Escalabilidad, trade-offs, diseno de sistemas, critica tecnica profunda	21 – 30

Indice de Preguntas

Bloque 1 — Comprension General

1. ¿Qué problema concreto resuelve Ticketia y por qué WhatsApp es el canal elegido en lugar de una app móvil propia?
2. Explica el concepto de multi-tenancy en tu proyecto: ¿cómo sabe el sistema a qué negocio pertenece cada mensaje que llega?
3. ¿Qué es el Model Context Protocol (MCP) y por qué lo has integrado en lugar de llamar directamente a las funciones de Python?
4. ¿Qué diferencia hay entre un agente reactivo y un agente proactivo en tu arquitectura? Pon un ejemplo concreto de cada uno.
5. Si un usuario envía una foto por WhatsApp, ¿cuál es el flujo completo desde que Twilio recibe la imagen hasta que el usuario obtiene respuesta?
6. Has elegido Flask sobre FastAPI. El CouncilManager usa asyncio dentro de una app Flask síncrona. ¿Cómo resuelves esa inconsistencia y qué implicaciones tiene?
7. ¿Por qué GPT-4o y no un modelo open-source como LLaMA o Mistral? ¿Has medido la diferencia de rendimiento en extracción de datos de tickets en español?
8. El scheduler original usaba la librería schedule. Lo has migrado a APScheduler con jobstore SQL. Explica exactamente qué problema resuelve esta migración y qué significa coalesce=True.
9. En el docker-compose.yml tienes cuatro servicios: db, mcp, web y scheduler. ¿Por qué el servicio MCP es independiente y no corre dentro del proceso de la app Flask?
10. ¿Qué es el WhatsAppWebhookDispatcher y por qué se ha diseñado como un patrón Dispatcher en lugar de poner toda la lógica directamente en el webhook?

Bloque 2 — Preguntas Tecnicas

11. El AgentExecutor hace dos llamadas a GPT-4o cuando hay tool calls. Explica por qué son necesarias dos llamadas y qué ocurre entre ambas.
12. El Consejo Estratégico tiene tres personas (El Socio, El Gestor, El Coach) sobre el mismo modelo GPT-4o. ¿Esto es realmente un sistema multi-agente? Defiende tu posición.
13. ¿Cómo decide el LLM qué herramienta usar? ¿Es lógica de código o es el propio modelo quien razona sobre ello?
14. El MarketingAgent siempre se ejecuta en un hilo secundario (background_tasks.py). ¿Por qué? ¿Qué pasaría si se ejecutara en el hilo principal de Flask?
15. ¿Qué estrategia de memoria conversacional usa el AgentExecutor? ¿Cuál es su limitación y cómo la mejorarías?
16. Has implementado rate limiting con Flask-Limiter. ¿Por qué el endpoint /login tiene un límite diferente al de /api/chat? ¿Qué ataque específico mitiga cada uno?
17. El aislamiento multi-tenant se hace filtrando por user_phone en el código de aplicación. ¿Qué pasaría si hubiera un bug en ese filtrado? ¿Qué mecanismo de defensa adicional añadirías en producción?
18. ¿Cómo evalúas la calidad de las respuestas de los agentes? ¿Qué métricas usa DeepEval y qué mide exactamente Answer Relevancy vs Faithfulness?
19. Tienes 30 tests en test_api_auth.py. El test test_agent_returns_text_response mockea OpenAI. ¿Qué valor tiene un test que nunca llama al LLM real? ¿Qué no puede detectar?
20. Las API keys están en .env. En producción con Kubernetes, ¿cómo gestionarías los secretos y por qué .env no es suficiente?

Bloque 3 — Preguntas Avanzadas

- 21.** Con 1.000 usuarios activos enviando mensajes simultáneamente por WhatsApp, ¿dónde están los cuellos de botella de tu arquitectura actual? Ordénalos por impacto.
- 22.** El ThreadPoolExecutor del scheduler tiene max_workers=4. ¿Qué ocurre si hay 100 negocios activos y el scheduler intenta ejecutar sus agentes al mismo tiempo?
- 23.** El servidor MCP SSE corre en el puerto 8001. Si el servicio mcp se cae mientras la app está procesando una llamada de herramienta, ¿qué ocurre exactamente en el código? ¿Hay fallback?
- 24.** Los documentos generados (PDFs, PPTs) se guardan en static/generated_docs/ dentro del contenedor, mapeado a un volumen Docker. ¿Por qué esto no escala en producción con múltiples instancias de la app?
- 25.** ¿Qué es un context processor en Flask y por qué has implementado inject_notifications() como uno en lugar de hacerlo en cada vista individualmente?
- 26.** El AgentExecutor usa las últimas 10 interacciones como historial. Si una conversación relevante ocurrió hace 15 mensajes, el agente no tiene acceso a ella. Diseña una solución usando embeddings que resuelva este problema.
- 27.** El Consejo Estratégico hace streaming via SSE al frontend. Si el servidor Flask tiene 2 workers de gunicorn y dos usuarios simultáneos lanzan un Council, ¿puede haber interferencia entre sus streams? ¿Hay race condition?
- 28.** Has usado session_transaction() en los tests para inyectar la sesión directamente sin pasar por el login. ¿Qué ventaja tiene esto y qué escenario de bug real NO detectaría comparado con hacer el login completo en el test?
- 29.** El GrantHunter busca subvenciones usando DuckDuckGo. Un tribunal podría argumentar que los resultados no son fiables porque el LLM puede alucinar al interpretar los resultados de búsqueda. ¿Cómo diseñarías un mecanismo de verificación de fuentes?
- 30.** Si tuvieras que eliminar la dependencia de OpenAI manteniendo la funcionalidad actual, ¿qué partes del sistema son sustituibles con modelos open-source y cuáles representarían una degradación significativa? Justifica técnicamente cada caso.

BLOQUE 1 — COMPRENSION GENERAL

Pregunta 1. [COMPRENSION]

¿Qué problema concreto resuelve Ticketia y por qué WhatsApp es el canal elegido en lugar de una app móvil propia?

Respuesta:

Ticketia automatiza la gestión administrativa y de negocio de autónomos y PYMEs españolas: captura y categoriza gastos (tickets/facturas via OCR), genera documentos comerciales (propuestas, presupuestos), detecta subvenciones relevantes y proporciona un asistente de IA accesible 24/7. WhatsApp se elige como canal principal porque tiene una penetración del 93% en España según Statista (2024), lo que significa adopción cero para el usuario: no instala nada nuevo. El flujo de enviar una foto del ticket desde WhatsApp es exactamente igual al que ya usa el usuario para comunicarse con clientes o proveedores, eliminando la fricción de aprendizaje.

- Tasa de adopción: ninguna app nueva tiene ese alcance en el segmento PYME español
- Canal ya integrado: el usuario ya tiene WhatsApp abierto durante su jornada laboral
- Multimodal por defecto: texto, audio, imagen y documentos sin fricciones adicionales
- Coste de desarrollo: construir apps iOS+Android nativas está fuera del alcance de un MVP académico

Trampa potencial: Si el tribunal pregunta '¿y por qué no Telegram?': Telegram tiene mejor API técnica pero ~15% de penetración en el segmento empresarial español frente al 93% de WhatsApp. La decisión es de mercado, no técnica.

Pregunta 2. [COMPRENSION]

Explica el concepto de multi-tenancy en tu proyecto: ¿cómo sabe el sistema a qué negocio pertenece cada mensaje que llega?

Respuesta:

Ticketia implementa multi-tenancy con esquema compartido (shared database, shared schema): todos los negocios comparten la misma base de datos PostgreSQL y el aislamiento se realiza a nivel de aplicación. Cada BusinessProfile tiene un campo user_phone único que actúa como identificador de tenant. Cuando llega un webhook de WhatsApp, el dispatcher extrae el número de destino del mensaje y hace una query filtrando por ese número. Todas las queries posteriores (tickets, documentos, citas, historial de chat) siempre incluyen el filtro WHERE user_phone = , garantizando que un negocio nunca ve datos de otro.

- Modelo más simple de los tres patrones multi-tenant (shared db > schema por tenant > db por tenant)
- Ventaja: una sola instancia de PostgreSQL, migraciones aplicadas a todos los tenants a la vez
- Desventaja: el aislamiento depende del código, no de la BD. Un bug de filtrado expondría datos
- Mejora para producción: Row Level Security (RLS) en PostgreSQL como segunda capa de defensa

```
SELECT * FROM tickets WHERE user_phone = '34600000001' -- filtro en CADA query
```

Pregunta 3. [COMPRENSION]

¿Qué es el Model Context Protocol (MCP) y por qué lo has integrado en lugar de llamar directamente a las funciones de Python?

Respuesta:

MCP es un estándar abierto definido por Anthropic para que los LLMs accedan a herramientas externas de forma estandarizada, independientemente del modelo o cliente que se use. En lugar de hardcodear las herramientas dentro del código del agente, el agente descubre en runtime las herramientas disponibles consultando al servidor MCP. Esto aporta tres ventajas concretas sobre llamadas directas a funciones Python: (1) desacoplamiento: el servidor de herramientas puede cambiar sin tocar el agente; (2) interoperabilidad: el mismo servidor MCP puede ser consumido por Claude Desktop, otros modelos o futuras versiones de la app; (3) estándar de industria: MCP está siendo adoptado por los principales proveedores de IA, lo que posiciona la arquitectura para el futuro.

- El servidor MCP expone 4 herramientas: `get_financial_summary`, `search_web`, `schedule_appointment`, `send_email_notification`
- El cliente MCP (`core/mcp_client.py`) usa transporte SSE en producción (proceso persistente) y stdio como fallback en desarrollo
- La selección de herramienta la hace el LLM en runtime basándose en la descripción de cada tool, no el código

Pregunta 4. [COMPRENSION]

¿Qué diferencia hay entre un agente reactivo y un agente proactivo en tu arquitectura? Pon un ejemplo concreto de cada uno.

Respuesta:

Un agente reactivo se activa cuando el usuario hace algo: envía un mensaje, sube una imagen o hace clic en el chat web. Su latencia objetivo es de segundos. Un agente proactivo se ejecuta de forma autónoma en background, sin intervención del usuario, según un calendario o trigger. Su ejecución puede tardar minutos porque el usuario no está esperando respuesta inmediata. La distinción arquitectónica clave es que los reactivos viven en el ciclo request/response de Flask mientras que los proactivos viven en el scheduler (proceso separado) y se comunican con el usuario a través de notificaciones o mensajes WhatsApp push.

- REACTIVO — ejemplo: el usuario envía 'Cuánto llevo gastado este mes' → AgentExecutor llama a `get_financial_summary` via MCP → responde en ~3 segundos
 - PROACTIVO — ejemplo: el GrantHunter se ejecuta cada lunes a las 09:00, busca en el BOE subvenciones para el sector del negocio con DuckDuckGo, y si encuentra algo relevante envía un WhatsApp al usuario sin que este haya pedido nada
 - El scheduler usa APScheduler con jobstore SQL para que los jobs proactivos sobrevivan a reinicios del servidor
-

Pregunta 5. [COMPRENSION]

Si un usuario envía una foto por WhatsApp, ¿cuál es el flujo completo desde que Twilio recibe la imagen hasta que el usuario obtiene respuesta?

Respuesta:

El flujo completo tiene 8 pasos. Twilio recibe la imagen en sus servidores y hace un HTTP POST al webhook configurado en /whatsapp. Flask recibe la petición y valida la firma HMAC de Twilio (cabecera X-Twilio-Signature) para verificar autenticidad. El WhatsAppWebhookDispatcher extrae el número de origen, destino y la URL de la imagen. Si el agente AdminRedactor está activo para ese negocio, llama a GPT-4o Vision con la URL de la imagen para clasificarla: ¿es un borrador de propuesta o un ticket de gasto? Si es ticket, se llama a process_ticket_image() que extrae importe, proveedor, fecha y categoría via OCR con GPT-4o Vision y lo guarda en la BD. Si es borrador, se pasa al AgentExecutor con la media_url para generar una propuesta PDF. Finalmente se envía la respuesta al usuario via API de Twilio/WhatsApp.

- Paso 1: Twilio POST → /whatsapp (validación firma HMAC)
- Paso 2: WhatsAppDispatcher detecta tipo de número (dedicado o central)
- Paso 3: AdminRedactor clasifica la imagen con GPT-4o Vision
- Paso 4a (ticket): process_ticket_image → OCR → guardado en BD → respuesta confirmación
- Paso 4b (borrador): AgentExecutor con media_url → genera PDF → envía por email y WhatsApp
- Tiempo total estimado: 3-8 segundos dependiendo de la latencia de OpenAI

BLOQUE 2 — PREGUNTAS TECNICAS

Pregunta 6. [TECNICA]

Has elegido Flask sobre FastAPI. El CouncilManager usa asyncio dentro de una app Flask síncrona. ¿Cómo resuelves esa inconsistencia y qué implicaciones tiene?

Respuesta:

El 95% de los endpoints son síncronos y Flask tiene un ecosistema más maduro para las integraciones necesarias (Flask-SQLAlchemy, Flask-Mail, Flask-Admin). FastAPI añadiría complejidad async que requeriría migrar el ORM a SQLAlchemy async y refactorizar todos los modelos. El Council usa AsyncOpenAI para el streaming SSE. La solución es ejecutar el loop async dentro del contexto síncrono de Flask usando asyncio.run() o un generador que el stream_with_context de Flask gestiona. La implicación es que durante una sesión del Council, el worker de gunicorn que la atiende queda bloqueado en el stream, no puede atender otras peticiones. Con 2 workers y 2 usuarios simultáneos haciendo Council, el tercero esperaría.

- Solución para producción: mover el Council a un microservicio FastAPI independiente
- Alternativa inmediata: aumentar el número de workers de gunicorn (-w 4 o más)
- La inconsistencia es una deuda técnica conocida y documentada, no un bug oculto

Trampa potencial: El tribunal puede preguntar: '¿Por qué no usaste directamente FastAPI para todo?'
Respuesta honesta: el panel de administración (Flask-Admin) y la integración de sesiones con SQLAlchemy hubieran requerido mucho más trabajo en FastAPI para el mismo resultado.

Pregunta 7. [TECNICA]

¿Por qué GPT-4o y no un modelo open-source como LLaMA o Mistral? ¿Has medido la diferencia de rendimiento en extracción de datos de tickets en español?

Respuesta:

La decisión se basa en tres factores medibles. Primero, precisión en OCR de documentos en español: las facturas españolas tienen formatos muy heterogéneos (escritura manual, sellos, IVA desglosado, número de factura en posiciones variables) y GPT-4o Vision supera consistentemente a los modelos open-source disponibles en 2024-2025 en esta tarea específica. Segundo, extracción estructurada: el sistema necesita extraer importe, proveedor, fecha, categoría fiscal y número de factura en un solo paso; GPT-4o lo hace con prompts simples mientras que los modelos open-source requieren fine-tuning específico. Tercero, el contexto académico no permite infraestructura GPU para servir modelos locales de 70B parámetros con latencia aceptable.

- Coste real: ~0.01€ por ticket procesado con GPT-4o Vision (input + output tokens)
- Riesgos: vendor lock-in, precio variable, latencia de red, cambios de API
- Mitigación: el cliente OpenAI está centralizado en core/clients.py como singleton, facilitando el reemplazo del proveedor sin modificar el código de los agentes
- En producción: se implementaría un router de LLMs (litellm) para fallback a Claude o Gemini

Pregunta 8. [TECNICA]

El scheduler original usaba la librería schedule. Lo has migrado a APScheduler con jobstore SQL. Explica exactamente qué problema resuelve esta migración y qué significa coalesce=True.

Respuesta:

La librería schedule almacena los jobs en memoria del proceso Python. Si el servidor se reinicia (por deploy, crash, mantenimiento), la próxima ejecución programada se pierde: el scheduler empieza de cero y los agentes proactivos pueden no ejecutarse en días. APScheduler con SQLAlchemyJobStore persiste cada job en una tabla de la base de datos (apscheduler_jobs). Al reiniciar, APScheduler lee los jobs de la BD y sabe exactamente cuándo debían haberse ejecutado. coalesce=True significa que si el servidor estuvo caído durante el tiempo de varias ejecuciones programadas, APScheduler las colapsará en una sola ejecución al recuperarse, en lugar de ejecutar todas las acumuladas en cascada, lo que podría saturar la API de OpenAI o generar notificaciones duplicadas para el usuario.

- misfire_grace_time=3600: si el job se retrasa menos de 1 hora (por carga del sistema), se ejecuta igualmente en lugar de descartarse
 - max_instances=1: nunca dos instancias del mismo job al mismo tiempo, evitando race conditions en la BD
 - ThreadPoolExecutor(max_workers=4): permite ejecutar agentes de hasta 4 usuarios en paralelo
-

Pregunta 9. [TECNICA]

En el docker-compose.yml tienes cuatro servicios: db, mcp, web y scheduler. ¿Por qué el servicio MCP es independiente y no corre dentro del proceso de la app Flask?

Respuesta:

El servidor MCP SSE debe ser un proceso HTTP persistente que acepta conexiones de larga duración. Si corriera dentro del proceso Flask, cada petición al servidor MCP compite por los workers de gunicorn con las peticiones normales de la app. Además, el servidor MCP puede ser consumido por múltiples clientes simultáneamente (la app web, el scheduler, herramientas externas como Claude Desktop) y esto requiere un proceso independiente. La arquitectura de microservicio también permite escalar el servidor MCP de forma independiente si el número de llamadas a herramientas crece, sin escalar innecesariamente los workers de la app Flask.

- El healthcheck de db con pg_isready garantiza que Postgres está listo antes de que web o scheduler intenten conectarse
- depends_on con condition: service_healthy evita el race condition clásico de arranque en Docker
- Los volúmenes uploads_data y generated_docs persisten los ficheros de usuario entre reinicios del contenedor

```
db healthcheck:
  test: [CMD-SHELL, pg_isready -U postgres -d ticketia_db]
  interval: 5s
  retries: 10
```

Pregunta 10. [TECNICA]

¿Qué es el WhatsAppWebhookDispatcher y por qué se ha diseñado como un patrón Dispatcher en lugar de poner toda la lógica directamente en el webhook?

Respuesta:

El Dispatcher encapsula la lógica de clasificación y enrutamiento de mensajes WhatsApp en una clase dedicada, separándola de la capa de transporte HTTP (webhooks.py). El webhook solo tiene una responsabilidad: recibir la petición de Twilio, validar la firma y delegar al Dispatcher. El Dispatcher tiene la responsabilidad de clasificar: ¿el mensaje va a un número dedicado del cliente o al número central de Ticketia? ¿Es audio, imagen o texto? ¿El usuario está registrado? Esta separación facilita los tests (se puede instanciar el Dispatcher con datos simulados sin levantar un servidor HTTP), la mantenibilidad (añadir un nuevo canal como Telegram solo requiere un nuevo Dispatcher) y la lectura del código.

- Patrón de diseño: Strategy + Chain of Responsibility adaptados al contexto de mensajería
 - Testabilidad: el Dispatcher se puede probar en unitarios pasando un objeto request mock
 - Extensibilidad: _handle_dedicated_client_number y _handle_ticketia_central_number son métodos privados independientes
 - La validación de firma HMAC de Twilio se hace en webhooks.py antes de llegar al Dispatcher
-

Pregunta 11. [TECNICA]

El AgentExecutor hace dos llamadas a GPT-4o cuando hay tool calls. Explica por qué son necesarias dos llamadas y qué ocurre entre ambas.

Respuesta:

El protocolo de function calling de OpenAI requiere dos llamadas porque el LLM no ejecuta las herramientas directamente: solo decide qué herramienta usar y con qué argumentos. Primera llamada: el LLM recibe el historial + system prompt + lista de tools disponibles. Si necesita información externa, devuelve un objeto `tool_calls` (no texto) con el nombre de la función y los argumentos en JSON. El código Python ejecuta la herramienta (query a BD, búsqueda web, etc.) y obtiene el resultado real. Segunda llamada: se añade el resultado de la herramienta al historial con `role='tool'` y se llama de nuevo al LLM para que sintetice ese resultado en lenguaje natural. Sin la segunda llamada, el usuario recibiría el JSON crudo del resultado de la herramienta en lugar de una respuesta conversacional.

- Primera llamada → LLM devuelve: `{tool_calls: [{name: 'get_financial_summary', args: {user_phone: '...'}}]}`
 - Python ejecuta: `resultado = get_financial_summary(user_phone='...')`
 - Segunda llamada → LLM recibe el resultado y genera: 'Este mes llevas 1.250€ en gastos, un 20% más que el mes pasado'
 - Si hay múltiples `tool_calls`, se ejecutan todas antes de la segunda llamada
-

Pregunta 12. [TECNICA]

El Consejo Estratégico tiene tres personas (El Socio, El Gestor, El Coach) sobre el mismo modelo GPT-4o. ¿Esto es realmente un sistema multi-agente? Defiende tu posición.

Respuesta:

Técnicamente es un sistema de role prompting o persona simulation sobre un único LLM, no un sistema multi-agente en el sentido estricto (modelos separados, comunicación asíncrona, memoria independiente). Académicamente, el término correcto es 'persona-based multi-agent simulation', un patrón bien documentado en la literatura de sistemas LLM. La defensa es que el valor arquitectónico no está en usar modelos distintos sino en el protocolo de debate estructurado: tres rondas con posiciones definidas (opinión → réplica → síntesis) que fuerza al LLM a considerar perspectivas contradictorias y reduce el sesgo de confirmación que tendría un único prompt genérico. El resultado práctico es superior a preguntar 'dame consejo sobre este tema' a un LLM sin estructura.

- Similitud con sistemas multi-agente reales: cada persona tiene system prompt propio, historial de intervenciones separado y área de expertise definida
- Diferencia con sistemas multi-agente reales: mismo modelo base, sin memoria persistente entre sesiones, sin comunicación asíncrona real
- Para una v2: usar modelos especializados (GPT-4o para El Socio, Claude Opus para El Gestor con su mejor razonamiento legal, Gemini para El Coach)
- Referencia académica: 'Society of Mind' (Minsky) y 'Communicative Agents for Software Development' (ChatDev, 2023)

Trampa potencial: El tribunal puede intentar desconcertarte diciendo 'entonces no es multi-agente'. La respuesta es: 'Correcto, es persona simulation, que es un subconjunto del paradigma multi-agente cuando el objetivo es la diversidad de perspectivas sobre un dominio de problema único.'

Pregunta 13. [TECNICA]

¿Cómo decide el LLM qué herramienta usar? ¿Es lógica de código o es el propio modelo quien razona sobre ello?

Respuesta:

Es el propio modelo quien razona. El código solo proporciona la lista de herramientas disponibles en formato JSON (nombre, descripción, parámetros con tipos) en la llamada a la API. GPT-4o lee las descripciones en lenguaje natural de cada herramienta y decide en runtime cuál es apropiada para el mensaje del usuario, qué argumentos extraer del contexto de la conversación y si necesita usar una, varias o ninguna. El código no tiene lógica condicional como 'si el usuario pregunta por gastos, llamar a `get_financial_summary`'. Esto es la esencia del paradigma de agentes LLM: el razonamiento sobre qué hacer está en el modelo, no en el código.

- La descripción de la herramienta es crítica: una descripción ambigua produce selecciones incorrectas
- Ejemplo de descripción efectiva: 'Get a summary of a user expenses based on their registered tickets. Use when the user asks about spending, expenses or financial data'
- El código en `tools.py` define el JSON schema de cada herramienta con tipos Python que se convierten a JSON Schema
- Si el LLM elige una herramienta incorrectamente, la solución es mejorar la descripción, no añadir lógica en el código

Pregunta 14. [TECNICA]

El MarketingAgent siempre se ejecuta en un hilo secundario (`background_tasks.py`). ¿Por qué? ¿Qué pasaría si se ejecutara en el hilo principal de Flask?

Respuesta:

La generación de contenido de marketing (imágenes DALL-E, presentaciones PPT, vídeos con Runway ML) puede tardar entre 30 segundos y varios minutos. HTTP tiene un timeout por defecto y gunicorn tiene un timeout configurado a 120 segundos. Si el MarketingAgent corriera en el hilo principal, el worker de gunicorn quedaría bloqueado durante toda la generación, sin poder atender otras peticiones. El navegador del usuario mostraría el spinner durante minutos y eventualmente un error de timeout. Al correr en `background_tasks.py` con `threading.Thread`, el endpoint responde inmediatamente con 'Generando contenido, te avisamos cuando esté listo' y el hilo de background procesa la tarea, guarda el resultado en la BD y notifica al usuario vía WhatsApp o notificación interna.

- Pattern UX: immediate acknowledgment + async processing + push notification cuando termina
 - Riesgo del threading simple: si el proceso Flask se reinicia, el hilo muere y la tarea se pierde
 - Mejora para producción: Celery con Redis como broker de tareas, que persiste las tareas y permite reintentos automáticos
 - El hilo necesita el contexto de Flask para acceder a la BD: se pasa con `current_app._get_current_object()`
-

Pregunta 15. [TECNICA]

¿Qué estrategia de memoria conversacional usa el AgentExecutor? ¿Cuál es su limitación y cómo la mejorarías?

Respuesta:

El AgentExecutor usa una ventana deslizante de las últimas N interacciones (configurable, por defecto 10 mensajes). En cada llamada, recupera los últimos 10 mensajes del historial de la BD (tabla chat_messages), los añade al contexto enviado al LLM y luego guarda el nuevo par usuario/asistente. Esta estrategia es $O(1)$ en coste por llamada pero $O(0)$ en memoria de largo plazo: el agente 'olvida' todo lo anterior a los 10 últimos mensajes.

- Limitación concreta: si el usuario configuró su negocio hace 3 semanas y ahora pregunta por ello, el agente no lo recuerda a menos que esté en el system prompt
- Mejora 1 — Memoria semántica: vectorizar el historial con embeddings (text-embedding-3-small), indexar en una BD vectorial (pgvector sobre PostgreSQL existente) y recuperar los K mensajes más similares semánticamente al mensaje actual, en lugar de los K más recientes
- Mejora 2 — Memoria episódica: guardar un resumen comprimido de cada sesión de conversación con el LLM y añadirlo al contexto como 'memoria de largo plazo'
- Mejora 3 — System prompt como memoria: el wizard de configuración ya captura información del negocio en el system_prompt del BusinessProfile, que se inyecta en cada llamada

Pregunta 16. [TECNICA]

Has implementado rate limiting con Flask-Limiter. ¿Por qué el endpoint /login tiene un límite diferente al de /api/chat? ¿Qué ataque específico mitiga cada uno?

Respuesta:

Son ataques de naturaleza diferente que requieren defensas distintas. El login tiene 5 intentos por minuto por IP para mitigar ataques de fuerza bruta: un atacante que intenta combinaciones de contraseña contra una cuenta conocida. El límite bajo hace que un ataque de diccionario se vuelva prohibitivamente lento (máximo 7.200 intentos por día desde una IP, vs millones por segundo sin limitación). El endpoint /api/chat tiene 30 peticiones por minuto para mitigar el abuso de la API de OpenAI: un usuario (legítimo o comprometido) que envía mensajes en bucle automatizado puede generar facturas de cientos de euros en horas. El límite más generoso permite un uso conversacional normal sin interrumpir la experiencia.

- Login: 5/min → anti-brute-force + anti-credential-stuffing
 - Chat: 30/min → anti-abuse de tokens OpenAI + anti-DDoS económico
 - Upload ticket/audio: 20/hora → límite natural del uso real (nadie escanea 20 tickets por hora)
 - Council stream: 10/hora → cada sesión de Council consume muchos tokens, límite protege el presupuesto
 - Video generation: 5/hora → Runway ML cobra por generación, límite crítico para control de costes
 - Flask-Limiter usa la IP del cliente como clave por defecto; en producción con load balancer habría que usar X-Forwarded-For
-

Pregunta 17. [TECNICA]

El aislamiento multi-tenant se hace filtrando por user_phone en el código de aplicación. ¿Qué pasaría si hubiera un bug en ese filtrado? ¿Qué mecanismo de defensa adicional añadirías en producción?

Respuesta:

Un bug de filtrado podría permitir que un usuario A acceda a los tickets, documentos o historial de chat de un usuario B. El riesgo más grave sería en la función run_agent si se pasara el phone_number incorrecto: el agente respondería con datos financieros de otro negocio. Para mitigarlo, la defensa adicional en producción sería Row Level Security (RLS) de PostgreSQL: una política a nivel de base de datos que impide físicamente que una sesión de BD lea filas que no pertenecen al tenant actual. Incluso si el código de aplicación tiene un bug, la BD rechazaría la query. Esto es defensa en profundidad: el código es la primera capa, la BD es la segunda.

- RLS en PostgreSQL: ALTER TABLE tickets ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON tickets USING (user_phone = current_setting('app.current_tenant'))
- La sesión Flask usa session['user_phone'] firmada criptográficamente con SECRET_KEY, lo que impide que el usuario manipule su propio tenant_id desde el navegador
- Auditoría: la tabla ActivityLog registra todas las acciones con el user_phone del actor, facilitando la detección forense de accesos cruzados
- Tests de seguridad: los 30 tests del fichero test_api_auth.py verifican que todos los endpoints devuelven 401 sin sesión activa

Pregunta 18. [TECNICA]

¿Cómo evalúas la calidad de las respuestas de los agentes? ¿Qué métricas usa DeepEval y qué mide exactamente Answer Relevancy vs Faithfulness?

Respuesta:

El pipeline LLMops en llmops/eval_agents.py usa DeepEval para evaluar respuestas de los agentes contra un gold standard de casos de uso reales. Answer Relevancy mide si la respuesta responde a lo que el usuario preguntó: una respuesta puede ser factualmente correcta pero no responder a la pregunta concreta (score < 0.7 indica problema). Faithfulness mide si la respuesta contiene solo información que puede ser inferida del contexto proporcionado, sin alucinaciones: si el agente afirma que el usuario gastó 1.500€ pero el contexto solo muestra 500€ de tickets, Faithfulness detecta la alucinación (score < 0.8 indica problema). Contextual Precision mide si el contexto recuperado (historial, system prompt) es el apropiado para la pregunta.

- Answer Relevancy: ¿la respuesta aborda la pregunta? (problema: respuestas genéricas o evasivas)
 - Faithfulness: ¿la respuesta está fundamentada en el contexto? (problema: alucinaciones del LLM)
 - Contextual Precision: ¿se usó el contexto correcto? (problema: historial irrelevante contaminando el contexto)
 - El gold standard (datasets/gold_standard.json) contiene pares input/expected_output para cada tipo de agente
 - En CI/CD: los tests de DeepEval se ejecutan antes de cada deploy para detectar regresiones en calidad
-

Pregunta 19. [TECNICA]

Tienes 30 tests en `test_api_auth.py`. El test `test_agent_returns_text_response` mockea OpenAI. ¿Qué valor tiene un test que nunca llama al LLM real? ¿Qué no puede detectar?

Respuesta:

Un test con mock de OpenAI verifica la lógica de orquestación del AgentExecutor: que se construye el contexto correctamente, que se guarda el historial en BD, que se usa el system prompt del negocio, que se manejan los errores de la API, que el flujo de `tool_calls` funciona. Lo que NO puede detectar: cambios en el comportamiento del modelo (degradación de calidad de respuestas), cambios en la API de OpenAI (nuevos campos, campos deprecados), problemas de rate limiting reales, y la calidad semántica de las respuestas. Para eso existe el pipeline de DeepEval, que evalúa contra el LLM real usando el gold standard. La combinación de ambos es el enfoque correcto: tests unitarios con mock para lógica de código + tests de evaluación LLMOps para calidad del modelo.

- Los mocks permiten ejecutar los tests sin coste económico y con velocidad determinista
- Los tests con mock garantizan que el código funciona; DeepEval garantiza que el sistema funciona
- Analogía: los tests unitarios de una calculadora verifican que el código de suma es correcto; la evaluación LLM verifica que el 'cerebro' da respuestas de calidad
- Test de integración con LLM real: se ejecutaría manualmente antes de releases importantes, no en cada commit

Trampa potencial: El tribunal puede preguntar: '¿Por qué no tienes más tests de integración con la API real?'

Respuesta: coste económico (cada test llama a la API de pago), no determinismo (el LLM puede dar respuestas diferentes cada vez), y lentitud (los tests tardarían minutos).

Pregunta 20. [TECNICA]

Las API keys están en .env. En producción con Kubernetes, ¿cómo gestionarías los secretos y por qué .env no es suficiente?

Respuesta:

.env no es suficiente en producción porque: el fichero existe en disco con permisos de lectura para todos los procesos del sistema, no hay rotación automática de secretos, no hay auditoría de quién accedió a qué secreto y cuándo, y no escala a múltiples pods de Kubernetes. El estándar de producción es un secrets manager: en Kubernetes se usarían Kubernetes Secrets (base) o mejor HashiCorp Vault con el Vault Agent Injector que monta los secretos como variables de entorno en el pod en runtime, sin que el valor esté nunca en disco. En AWS se usaría AWS Secrets Manager con rotación automática. La diferencia clave es que el secreto se inyecta en el proceso en tiempo de ejecución y nunca toca el sistema de ficheros.

- Kubernetes Secrets: `kubectl create secret generic ticketia-secrets --from-literal=OPENAI_API_KEY=sk-...`
- Vault Agent: el agente de Vault se ejecuta como sidecar en el pod e inyecta los secretos como env vars al arrancar
- Rotación automática: AWS Secrets Manager puede rotar las credenciales de DB automáticamente sin downtime
- Auditoría: cada acceso a un secreto queda registrado con timestamp, pod y usuario IAM
- En el contexto del TFM: .env + .gitignore es el estándar aceptable para desarrollo/demo

BLOQUE 3 — PREGUNTAS AVANZADAS

Pregunta 21. [AVANZADA]

Con 1.000 usuarios activos enviando mensajes simultáneamente por WhatsApp, ¿dónde están los cuellos de botella de tu arquitectura actual? Ordénalos por impacto.

Respuesta:

De mayor a menor impacto: Primero, la API de OpenAI: cada mensaje activa al menos una llamada a GPT-4o que tarda 1-5 segundos. Con 1.000 mensajes simultáneos y rate limits de OpenAI (tier 1: ~60 req/min), se generaría una cola masiva. Segundo, los workers de gunicorn: con 2 workers cada sesión activa de Council bloquea un worker durante 30-60 segundos. Tercero, la base de datos PostgreSQL: 1.000 conexiones simultáneas pueden saturar el pool de conexiones de SQLAlchemy. Cuarto, el servidor MCP SSE: un solo proceso para todas las llamadas a herramientas. Quinto, el scheduler: ThreadPoolExecutor con 4 workers no escala para cientos de usuarios.

- Solución #1 (OpenAI): implementar una cola de mensajes (Redis + Celery) con priorización y rate limiting inteligente hacia la API
 - Solución #2 (workers): aumentar workers de gunicorn + separar el Council a un servicio async con FastAPI
 - Solución #3 (BD): PgBouncer como connection pooler entre la app y PostgreSQL
 - Solución #4 (MCP): múltiples instancias del servidor MCP detrás de un load balancer
 - Solución #5 (scheduler): migrar a Celery Beat con Redis como broker
-

Pregunta 22. [AVANZADA]

El ThreadPoolExecutor del scheduler tiene max_workers=4. ¿Qué ocurre si hay 100 negocios activos y el scheduler intenta ejecutar sus agentes al mismo tiempo?

Respuesta:

Con max_workers=4 y 100 negocios, APScheduler crea una cola interna. Los primeros 4 negocios se procesan en paralelo en 4 hilos. Los otros 96 esperan en la cola de APScheduler hasta que algún hilo queda libre. Cada agente proactivo puede tardar 10-60 segundos (implica llamadas a OpenAI y búsquedas web). En el peor caso, el último negocio esperaría $100/4 \times 60s = 25$ minutos desde las 09:00 hasta recibir su análisis diario. Esto es aceptable para una tarea diaria no urgente, pero no para un sistema de tiempo real. El parámetro misfire_grace_time=3600 garantiza que aunque tarde, el job se ejecuta si el retraso es menor de 1 hora.

- Solución de escala: aumentar max_workers a $\min(32, \text{os.cpu_count()}) + 4$ para I/O-bound tasks
- Solución de arquitectura: migrar a Celery con múltiples workers en pods Kubernetes separados
- La razón de 4 workers en el MVP: en un VPS pequeño (1-2 CPU), más hilos concurrentes causarían thrashing por el GIL de Python para las partes CPU-bound
- Monitorización: APScheduler registra tiempos de ejecución que se pueden exportar a Prometheus/Grafana

Pregunta 23. [AVANZADA]

El servidor MCP SSE corre en el puerto 8001. Si el servicio mcp se cae mientras la app está procesando una llamada de herramienta, ¿qué ocurre exactamente en el código? ¿Hay fallback?

Respuesta:

En core/mcp_client.py, el método _run_with_sse() está envuelto en un bloque try/except. Si la conexión SSE falla (ConnectionRefusedError, timeout, etc.), el except captura la excepción, imprime un log de warning y llama a _run_with_stdio() como fallback. El fallback stdio lanza mcp_server.py como subprocess usando el mismo Python del sistema, ejecuta el loop de herramientas y retorna. El usuario no percibe ningún error; puede haber una latencia adicional de ~500ms por el fork del proceso. Si también el fallback stdio falla (por ejemplo, mcp_server.py tiene un error de import), el except externo del _run_with_stdio captura el error y devuelve un mensaje de error al usuario.

- Cadena de degradación: SSE conectado → SSE caído → stdio fallback → error controlado al usuario
- El restart: unless-stopped en docker-compose garantiza que el servicio MCP se reinicia automáticamente si cae
- Mejora: añadir un circuit breaker (librería pybreaker) para no intentar conectar al SSE si ha fallado N veces en los últimos X segundos
- Monitorización: el log '[MCP-SSE] Error conectando...' debería enviarse a un sistema de alertas (Sentry, Datadog)

```
_run_with_sse falla → except captura → log warning
→ _run_with_stdio() como fallback
_run_with_stdio falla → except captura → return 'Error del sistema...'
```


Pregunta 24. [AVANZADA]

Los documentos generados (PDFs, PPTs) se guardan en static/generated_docs/ dentro del contenedor, mapeado a un volumen Docker. ¿Por qué esto no escala en producción con múltiples instancias de la app?

Respuesta:

Con un solo contenedor y un volumen Docker local, funciona. Con 3 réplicas del servicio web (3 pods en Kubernetes), cada pod tiene su propio sistema de ficheros. Un PDF generado en el pod A se guarda en el volumen del pod A. Cuando el usuario solicita descargarlo, puede ser atendido por el pod B que no tiene ese fichero, resultando en un 404. El problema es que el almacenamiento de ficheros no es compartido entre réplicas.

- Solución inmediata: un volumen NFS compartido montado en todos los pods (funciona pero es un single point of failure)
- Solución correcta: almacenamiento de objetos (S3, Google Cloud Storage, Azure Blob). Los ficheros se suben a S3 después de generarlos y se sirven desde la URL de S3 o un CDN
- Cambio de código necesario: en document.py y marketing_agent.py, reemplazar la escritura a disco local por una llamada a boto3 (AWS SDK) para subir a S3
- El docker-compose.yml ya usa volúmenes Docker nombrados (uploads_data, generated_docs) que son la primera mejora: los ficheros sobreviven al recrear el contenedor, aunque siguen sin ser compartidos entre réplicas

Pregunta 25. [AVANZADA]

¿Qué es un context processor en Flask y por qué has implementado inject_notifications() como uno en lugar de hacerlo en cada vista individualmente?

Respuesta:

Un context processor en Flask es una función decorada con @app.context_processor que se ejecuta automáticamente antes de renderizar cualquier template Jinja2 y añade variables al contexto de la plantilla sin que la vista tenga que pasarlas explícitamente. inject_notifications() consulta la BD para contar las notificaciones no leídas del usuario en sesión y las inyecta como unread_notifications_count disponible en todos los templates. La alternativa de hacerlo en cada vista sería copiar la misma query en las 15 rutas que renderizan templates, violando el principio DRY. Si la lógica cambia (por ejemplo, filtrar notificaciones por tipo), habría que cambiarlo en 15 sitios en lugar de uno.

- El context processor solo ejecuta la query si hay un usuario en sesión (if 'user_phone' in session), evitando queries innecesarias en páginas públicas
 - La navbar (base.html) muestra el badge de notificaciones usando directamente la variable {{ unread_notifications_count }}
 - Rendimiento: la query es un COUNT simple con índice sobre (user_phone, is_read), $O(\log n)$
 - Alternativa considerada: usar AJAX para cargar el conteo de forma asíncrona, eliminando la query síncrona del ciclo de render
-

Pregunta 26. [AVANZADA]

El AgentExecutor usa las últimas 10 interacciones como historial. Si una conversación relevante ocurrió hace 15 mensajes, el agente no tiene acceso a ella. Diseña una solución usando embeddings que resuelva este problema.

Respuesta:

La solución es Retrieval-Augmented Memory (RAM): en lugar de recuperar los N últimos mensajes, recuperar los K mensajes semánticamente más relevantes para el mensaje actual. Implementación: al guardar cada ChatMessage en BD, también se calcula su embedding con text-embedding-3-small de OpenAI (1536 dimensiones, 0.02€ por millón de tokens) y se guarda en una columna vector de PostgreSQL habilitando la extensión pgvector. En cada llamada al AgentExecutor, se calcula el embedding del mensaje actual y se hace una búsqueda ANN (approximate nearest neighbor) con pgvector para recuperar los K mensajes con mayor similitud coseno. Estos K mensajes se añaden al contexto junto con los últimos 3 mensajes (para mantener coherencia conversacional inmediata).

- Instalación: CREATE EXTENSION vector; ALTER TABLE chat_messages ADD COLUMN embedding vector(1536);
- Búsqueda: SELECT content FROM chat_messages WHERE user_phone=? ORDER BY embedding <=> \$query_embedding LIMIT 5
- Coste: ~0.0004€ por mensaje guardado (embedding) + ~0.0004€ por búsqueda, totalmente asumible
- pgvector ya está disponible en el PostgreSQL del docker-compose sin instalar nada adicional en versiones recientes
- Mejora adicional: guardar embeddings de sesiones completas (resúmenes) además de mensajes individuales

```
# En history.py
from openai import OpenAI
client = OpenAI()

def get_relevant_history(user_phone, current_message, k=5):
    embedding = client.embeddings.create(
        input=current_message,
        model='text-embedding-3-small'
    ).data[0].embedding
    # pgvector ANN search
    return db.session.execute(
        'SELECT content, role FROM chat_messages'
        ' WHERE user_phone=:phone'
        ' ORDER BY embedding <=> :emb LIMIT :k',
        {'phone': user_phone, 'emb': str(embedding), 'k': k}
    ).fetchall()
```

Pregunta 27. [AVANZADA]

El Consejo Estratégico hace streaming via SSE al frontend. Si el servidor Flask tiene 2 workers de gunicorn y dos usuarios simultáneos lanzan un Council, ¿puede haber interferencia entre sus streams? ¿Hay race condition?

Respuesta:

No hay interferencia entre streams de distintos usuarios porque cada petición HTTP tiene su propio scope de request en Flask. El `stream_with_context()` de Flask garantiza que el generador de eventos está vinculado al contexto de la petición original. Cada worker de gunicorn atiende una petición de forma aislada: worker 1 tiene el generator del usuario A, worker 2 tiene el generator del usuario B. Los datos de la sesión (`user_phone`, `business profile`) se leen al inicio de cada petición y se mantienen en el scope local de esa petición. El único riesgo de race condition sería en la escritura a BD si dos sesiones del mismo usuario se ejecutan simultáneas, pero el rate limit de 10/hora lo hace prácticamente imposible.

- El problema real con 2 workers: si un tercer usuario intenta abrir el Council, su petición espera hasta que uno de los dos workers termine el stream en curso (que puede durar 1-2 minutos)
 - Solución: gunicorn con worker class `eventlet` o `gevent` (async workers) que permiten concurrencia en un solo proceso mediante coroutines
 - Alternativa arquitectónica: separar el Council a un servicio FastAPI con workers async nativos donde una sola instancia puede manejar cientos de streams simultáneos
 - El SSE (`text/event-stream`) mantiene la conexión HTTP abierta: el worker está ocupado durante todo el debate, no solo durante los picos de CPU
-

Pregunta 28. [AVANZADA]

Has usado session_transaction() en los tests para inyectar la sesión directamente sin pasar por el login. ¿Qué ventaja tiene esto y qué escenario de bug real NO detectaría comparado con hacer el login completo en el test?

Respuesta:

La ventaja de session_transaction() es la velocidad y el aislamiento: no depende de que el endpoint de login funcione correctamente, no hace queries a la BD para validar credenciales y no ejecuta el hashing de bcrypt (que es intencionalmente lento). Los tests de endpoints autenticados son rápidos y deterministas. El escenario de bug que NO detecta es cualquier bug en el proceso de login en sí que afecte al estado de la sesión: por ejemplo, si el login no llamara a session.permanent = True correctamente, los tests con session_transaction() no lo descubrirían porque la sesión se inyecta ya con el estado correcto. También quedaría sin detectar si la cookie de sesión no se genera con los flags correctos (HttpOnly, SameSite) porque session_transaction() bypassa la generación de cookies.

- Detecta: que los endpoints requieren ciertos campos en sesión para funcionar
 - Detecta: que los endpoints manejan correctamente los datos del usuario en sesión
 - NO detecta: bugs en el flujo de login (hashing, generación de sesión, flags de cookie)
 - NO detecta: bugs en el logout (que la sesión se limpie correctamente)
 - La cobertura completa requiere AMBOS: session_transaction() para endpoints + tests de login completo como en TestAuthentication
 - Analogía: es como probar que una cerradura abre con la llave correcta pero no probar que la llave se fabrica bien
-

Pregunta 29. [AVANZADA]

El GrantHunter busca subvenciones usando DuckDuckGo. Un tribunal podría argumentar que los resultados no son fiables porque el LLM puede alucinar al interpretar los resultados de búsqueda. ¿Cómo diseñarías un mecanismo de verificación de fuentes?

Respuesta:

El problema es real: DuckDuckGo devuelve snippets de texto y el LLM puede interpretar erróneamente o extrapolar información no presente en el resultado. El mecanismo de verificación tendría tres capas. Primera capa: source validation, filtrar resultados de búsqueda para aceptar únicamente URLs de dominios de confianza (boe.es, cdti.es, red.es, comunidad de madrid, etc.) usando una whitelist mantenida. Segunda capa: grounding estricto en el prompt del LLM usando la técnica de instrucción de cita obligatoria: el LLM solo puede afirmar información que esté literalmente en el snippet, y debe incluir la URL de origen para cada afirmación. Tercera capa: verificación post-generación con un segundo LLM call que compara la respuesta generada con los snippets originales y detecta afirmaciones no fundamentadas.

- Whitelist de dominios oficiales: boe.es, infosubvenciones.es, cdti.es, comunidad*.es, ayuntamiento*.es
 - Prompt de grounding: 'Only include information that is explicitly stated in the search results. For each claim, cite the exact URL. If you cannot find information in the results, say so explicitly.'
 - Verificación con DeepEval: la métrica Faithfulness ya mide si las afirmaciones están fundamentadas en el contexto, exactamente este problema
 - Mejora arquitectónica: en lugar de DuckDuckGo, usar la API oficial del BOE (boletinoficial.gob.es/api) que devuelve datos estructurados sin ambigüedad semántica
 - Notificación al usuario: incluir siempre el enlace fuente en el mensaje de WhatsApp para que el usuario pueda verificar directamente
-

Pregunta 30. [AVANZADA]

Si tuvieras que eliminar la dependencia de OpenAI manteniendo la funcionalidad actual, ¿qué partes del sistema son sustituibles con modelos open-source y cuáles representarían una degradación significativa? Justifica técnicamente cada caso.

Respuesta:

La respuesta requiere analizar cada uso de OpenAI por separado. Sustituibles sin degradación significativa: el chat conversacional general (El Socio, El Gestor, El Coach del Council) se puede migrar a LLaMA 3.1 70B o Mistral Large que tienen capacidad conversacional comparable en español. La transcripción de audio (Whisper) es directamente reemplazable por el modelo open-source Whisper de OpenAI que corre localmente sin API, idéntica calidad. Sustituibles con degradación moderada: la generación de system prompts en el wizard y los agentes proactivos (GrantHunter, Networker) funcionarían con modelos open-source aunque con peor calidad en razonamiento complejo. Degradación significativa: el OCR de tickets y facturas (GPT-4o Vision) es donde la diferencia es mayor. Los modelos open-source de visión disponibles en 2025 (LLaVA, Qwen-VL) tienen peor rendimiento en extracción estructurada de documentos financieros en español con escritura manual. La generación de imágenes (DALL-E 3) es sustituible por Stable Diffusion pero con menor calidad en prompts complejos.

- Sustituible sin degradación: Whisper (modelo local), chat conversacional (LLaMA 3.1 70B)
- Sustituible con degradación moderada: agentes proactivos (razonamiento), generación de texto de marketing
- Degradación significativa: GPT-4o Vision para OCR de tickets (crítico para la función core del producto)
- Infraestructura necesaria: GPU A100 80GB para servir LLaMA 70B con latencia aceptable (~2-3s), coste ~\$2.5/hora en AWS
- Estrategia de migración: litellm como capa de abstracción sobre el cliente OpenAI, permite cambiar de proveedor en core/clients.py sin modificar el código de los agentes
- Veredicto: la dependencia más irremplazable es GPT-4o Vision para el OCR. Todo lo demás es sustituible con esfuerzo de ingeniería.

Trampa potencial: Esta es la pregunta más difícil del bloque. Demuestra que entiendes los trade-offs reales, no solo los conceptos teóricos. Un tribunal que hace esta pregunta quiere ver pensamiento crítico, no una respuesta de marketing sobre open-source.

Documento preparado para la defensa del TFM
Ticketia — Plataforma de IA para la Gestion Empresarial Automatizada
2026 | Trabajo Fin de Master